GLYPHOS — Phase 0 Report

Scaffold, Experiment Ledger, Reproducibility Infrastructure

Waiz Khan
JHU CLSP — advisor: Philipp Koehn

August 1, 2026 stamp: phase0-v1

1 Mission and hard constraints

GLYPHOS is a from-scratch pipeline for visual modeling, restoration, translation, and constrained decipherment of ancient scripts, validated by *artificial decipherment*: hide the key of an already-deciphered script, run the pipeline, and measure recovery. It combines the Salesky/Koehn pixel-representation lineage, the Luo–Barzilay constrained-decipherment lineage, and a quant-derived statistical epistemics layer.

Four constraints are enforced by tooling, not convention:

1. **No pretrained model weights, anywhere.** A component pretrained on the open web may have seen Ugaritic or Linear B scholarship; any decipherment it produces is void (lookahead bias). A CI gate greps the sealed tree for pretrained-loading APIs; the (currently empty) `contrib/openbook/` directory is the only exclusion. Pretrained *datasets* are allowed; pretrained *models* are not.
2. **Ledger before results.** Every run — including failed, abandoned, and smoke runs — is registered with a non-empty hypothesis *before* results exist, and closed with exactly one terminal event.
3. **Locked test sets.** Every read of a path under `data/**/test/` is audit-logged with the active run id.
4. **Reproducibility.** Explicit seeds (3+ for headline numbers), strict YAML→dataclass configs, content-hashed data and split versions recorded per run.

2 What was built

- **Scaffold:** `uv` project (Python 3.12), ruff, pytest, Makefile, GitHub Actions CI; package layout for all eight phases with loud stubs where later phases will land.
- **Experiment ledger** (`glyphos/ledger/`): append-only JSONL with a two-event lifecycle (`register`→`complete`). Registration requires the hypothesis and computes the family's running variant count (`n_variants_tried_so_far_in_this_family`) from the ledger itself, so the multiple-testing count cannot be curated after the fact. A context manager guarantees crashed runs are recorded as `failed`. CLI: `glyphos-ledger {report,list,show,register,complete}`; the report shows, per family, the number of variants tried, the full metric distribution (never

just the max), and rank stability across seeds (mean pairwise Kendall τ_a between per-seed config rankings).

- **Seed management:** one call seeds python/numpy/torch; `require_seeds` enforces the 3-seed headline policy; `derive_seed` gives stable component-level seeds.
- **Test-set guard:** patches both `builtins.open` and `io.open` (covering `pathlib`); reads under `data/**/test/` are appended to an audit log with timestamp, run id, path, and caller. Log-only in Phase 0; enforcement arrives with frozen-split manifests in Phase 1.
- **Hard-constraint gate:** `make check-no-pretrained` fails the build on `from_pretrained`, `AutoModel*`, `torch.hub`, hub-download APIs, and friends; tested in both directions (clean passes, planted violation fails, openbook exempt).
- **Report system:** `make reports` rebuilds *all* LaTeX reports in one pass (tectonic 0.17; no TeX previously existed on the dev machine) and a freshness gate verifies each PDF embeds its source's `\ReportStamp` and is not older than the `.tex`.

3 Smoke pipeline result

`make smoke` runs the smallest real instance of the artificial- decipherment protocol end to end, in the exact lifecycle shape every later phase must follow: strict config → guard install → seeding → data generation with content hashes → ledger registration (hypothesis first) → fit on train → select on valid → score on the locked test split (audited) → terminal ledger event → family report. The task: recover a hidden 12-letter substitution key over a Zipfian toy language by frequency-rank matching, under a document-held-out split.

metric (run `20260802-002751-6a0a99`, seed 1337)	value
key_accuracy (12-symbol hidden key)	1.000
valid_token_accuracy	1.000
test_token_accuracy	1.000
audited locked-test reads	1
wall time (budget: 300 s)	0.02 s

Table 1: Phase-0 smoke: infrastructure demonstration, not science. The pipeline asserts its own guard-audit entry and a key-accuracy regression floor of 0.5.

4 Verification

49 pytest tests (0.24 s), all state-isolated: ledger lifecycle, corruption handling, variant counters, report distributions and rank stability, both guard patch routes, seeds, hashing, strict configs, toy-corpus determinism and split disjointness, frequency-matching correctness, CLI round-trip, and the no-pretrained gate. `make check` (lint + tests + gate + smoke) is green.

5 Next: Phase 1

Ingest TLA (all HuggingFace subsets, with provenance), Coptic SCRIPTORIUM, LogogramNLP, and Ugaritic+Hebrew; produce the census table; implement the six split schemes (random, document-

held-out, dedup, period, site, sign-held-out); hash and freeze all test partitions and extend the guard with freeze enforcement. Known gaps (DĀMOS/LiBER access, Ugaritic source location) are filed in `docs/data_gaps.md`.